Reviewer Pack — Minimal Order of Monoidal Categorification and SAT Clause Su…

Entry 4dadd596457d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:15:12 UTC

Minimal Order of Monoidal Categorification and SAT Clause Subset Entropy Correlation

Entry ID: `4dadd596457d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 02:15:12 UTC

1. Conjecture

Field A (mathematical branch): Monoidal Category Theory **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

> For every conjunctive normal form (CNF) formula with n clauses, the minimal order of a monoidally categorified version of its clause set is linearly correlated with its conditional entropy over all possible subsets of clauses, such that O(n^c) ≤ Entropy(SATClauseSubset(CNF)) ≤ O(n^{2c}) for some constant c.

Rationale (proposer's reasoning):

> Monoidal categorification provides a structured framework to encode combinatorial objects like clause sets in terms of algebraic structures. The entropy of a SAT clause subset captures the information content of the set, which could be influenced by the categorical encoding. This correlation might reveal new insights into the complexity of satisfiability problems.

Taxonomy category: `CATEGORIFICATION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f17c4ce62a9c2e82`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson's correlation coefficient between minimal order and conditional entropy is within the range O(n^c) to O(n^{2c}) for some constant c, with no seed producing a correlation outside this range.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Monoidal Category Theory" AND "Boolean Satisfiability" AND CNF - "categorification" AND monoidally AND "SAT clause subset entropy" - "minimal order" AND categorification AND conditional entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
            clauses.append(clause)
        return clauses

    def monoidal_categorify(clauses):
        # Simplified categorification: count unique literals
        return len(set(lit for clause in clauses for lit in clause))

    def entropy(subset):
        if not subset:
            return 0
        p = Fraction(len(subset), len(clauses))
        return -p * math.log2(p)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        clauses = generate_cnf(n)
        min_order = monoidal_categorify(clauses)

        subset_entropies = []
        for r in range(1, len(subset) + 1):
```

```
            for subset in itertools.combinations(subset, r):
                subset_entropies.append(entropy(subset))

        if not subset_entropies:
            continue

        avg_entropy = sum(subset_entropies) / len(subset_entropies)
        results.append({
            "n": n,
            "min_order": min_order,
            "avg_entropy": avg_entropy
        })

    if not results:
        return {
            "metric_name": "Entropy",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No valid subsets found"
        }

    min_order = min(result["min_order"] for result in results)
    avg_entropy = sum(result["avg_entropy"] for result in results) / len(results)

    # Pearson's correlation coefficient
    n = len(results)
    x_sum = sum(result["min_order"] for result in results)
    y_sum = sum(result["avg_entropy"] for result in results)
    xy_sum = sum(result["min_order"] * result["avg_entropy"] for result in results)
    xx_sum = sum(result["min_order"] ** 2 for result in results)
    yy_sum = sum(result["avg_entropy"] ** 2 for result in results)

    numerator = n * xy_sum - x_sum * y_sum
    denominator = math.sqrt((n * xx_sum - x_sum ** 2) * (n * yy_sum - y_sum ** 2))

    if denominator == 0:
        return {
            "metric_name": "Entropy",
            "metric_value": 0,
            "instances_tested": n,
            "n_max": max(result["n"] for result in results),
            "conjecture_holds": False,
            "counterexample": "Denominator is zero"
        }

    correlation = numerator / denominator

    return {
        "metric_name": "Entropy",
        "metric_value": correlation,
        "instances_tested": n,
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": -1 <= correlation <= 1,
        "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE No valid trials found")
        sys.exit(0)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if -1 <= result["metric_value"] <= 1) / len(res

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation outside bounds\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE Fewer than 80% seeds support the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cf8291b4.py", line 111, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cf8291b4.py", line 46, in run_trial
    for r in range(1, len(subset) + 1):
                          ^^^^^^
UnboundLocalError: cannot access local variable 'subset' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the correlation coefficient could not be calculated and compared to the conjectured range. | next: Investigate the cause of the crash in the test code and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18178
2	propose	ollama_remote	glm4:latest	0	0	12657
3	propose	ollama_remote	glm4:latest	0	0	14047
4	preregistration	ollama_remote	glm4:latest	0	0	9227
5	novelty	ollama_remote	glm4:latest	0	0	8721
6	novelty	ollama_remote	glm4:latest	0	0	9050
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13884
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8416
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8843
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12494
11	judge	ollama_remote	glm4:latest	0	0	11731

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 127249 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/4dadd596457d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4dadd596457d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4dadd596457d.tar.gz` (if generated)